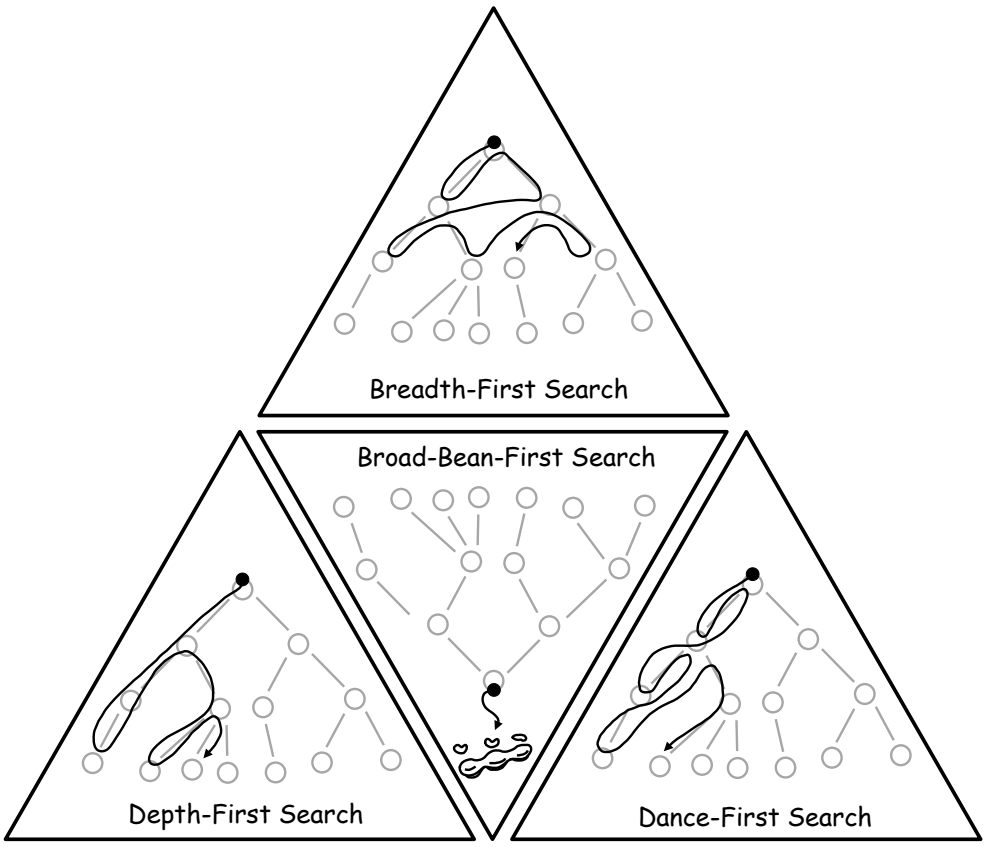
Università degli Studi di Perugia

Dipartimento di Matematica e Informatica

Corso di laurea triennale in Informatica

Algoritmi e Strutture Dati

Esercizi Risolti



F. Coró L. Palazzetti S. Bonacorsi

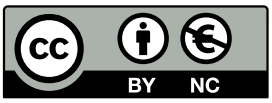
A cura di:
C. M. Pinotti

Tabella dei Contenuti

1 Algoritmi su Alberi **1**
1.1 Alberi Binari di Ricerca 1

2 Algoritmi su Grafi **5**
2.1 Visitare i Grafi 5
2.2 Operazioni sui Grafi 14
2.3 Cammini Minimi da Sorgente Singola 17
2.4 Alberi di Copertura 21
2.5 Cammini Minimi tra Tutte le Coppie 24

Quest'opera è distribuita con licenza Creative Commons "Attribuzione – Non commerciale 4.0 Internazionale".



Capitolo 1

Algoritmi su Alberi

1.1 Alberi Binari di Ricerca

Esercizio 1.1.1 #AlberoBinarioRicerca

Dato un albero binario di ricerca B i cui nodi contengono degli interi, restituire il successore dell'elemento minimo memorizzato in B. Descrivere un algoritmo che risolva il problema, darne lo pseudocodice e studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: La peculiarità di questo esercizio è che x è il minimo in T. Per trovare il minimo in un albero di ricerca T, si scorre il cammino più a sinistra dell'albero T. Una volta trovato il minimo nel nodo x, il suo successore è il minimo del sotto-albero di destra se $x.right \neq$ NULL, oppure il padre ($x.p$). Nella procedura generale, quando x è un nodo generico e non ha figlio dx, si segue a ritroso il cammino dal nodo verso la radice fino a scoprire che il nodo su cui ci troviamo è figlio sinistro di suo padre. In questo caso, poiché x è il minimo in T, la condizione è subito verificata dal padre di x.

Pseudocodice:

```
Successore_Minimo(Nodo x)
while x.left ≠ NULL do
    x = x.left // Cerchiamo il minimo
// Vediamo se il successore è nel sotto-albero dx o il padre
if x.right ≠ NULL then
    x = x.right
    while x.left ≠ NULL do
        x = x.left
    return: x
else
    return: x.p

main()
// Controlliamo se l'albero non è vuoto
if T ≠ NULL then
    return: SUCCESSORE_MINIMO(T)
```

Complessità: $O(h)$

Esercizio 1.1.2 #AlberoBinarioRicerca #Correttezza

Sia T un albero binario di ricerca, progettate un algoritmo per ordinare in senso crescente le chiavi memorizzate in T.
Discutete la correttezza della soluzione proposta.
Discutete la complessità in tempo della soluzione proposta.

Elaborazione Soluzione: Dalla definizione di albero binario di ricerca, per ordinare i valori nei nodi di un albero binario di ricerca è sufficiente effettuare una visita in-order dell'albero.

Pseudocodice n.1: Caso in cui ordiniamo in senso crescente.

```
In-Order(Nodo x)
if x ≠ NULL then
    In-Order(x.left)
    STAMPA(x)
    In-Order(x.right)
```

Pseudocodice n.2: Caso in cui ordiniamo in senso decrescente.

```
In-Order(Nodo x)
if x ≠ NULL then
    In-Order(x.right)
    STAMPA(x)
    In-Order(x.left)
```

Complessità:La complessità è $\Theta(n)$, poiché tutti ed n i nodi devono essere visitati.

Esercizio 1.1.3 #AlberoBinarioRicerca

Il fattore di bilanciamento $b(v)$ di un nodo interno v di un albero binario T è definito come la differenza fra le altezze dei sotto-alberi radicati nel figlio sinistro e destro di v.
Progettare in pseudocodice un algoritmo per calcolare $b(v)$, $\forall v \in T$.
Discutere la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Supponendo che i nodi dell'albero possano memorizzare sia la loro altezza che il loro fattore di bilanciamento, realizziamo un algoritmo che calcola le altezze dei nodi, per poi memorizzarle nel nodo. Poi modifichiamo una visita in-order (andava bene una visita qualsiasi) affinché per ogni nodo se ne imposti il fattore di bilanciamento conoscendo le altezze dei suoi sotto-alberi.

Pseudocodice:

```
Altezza(Nodo x)
if x ≠ NULL then
    x.altezza = 1 + MAX(altezza(x.left), altezza(x.right))
else
    return: 0

In-order-fat-bil(Nodo x)
if x ≠ NULL then
    In-order-fat-bil(x.left)
    if x.left ≠ NULL then
        if x.right ≠ NULL then
            x.fatBil = |x.left.altezza − x.right.altezza|
        else
            x.fatBil = x.left.altezza
    else
        if x.right ≠ NULL then
            x.fatBil = x.right.altezza
        else
            x.fatBil = 0
    In-order-fat-bil(x.right)
```

Lancio dell'algoritmo:

Main()
// Controlliamo se l'albero non è vuoto
if $T \neq$ NULL **then**
  ALTEZZA(T)
  IN-ORDER-FAT-BIL(T)

Complessità: L'algoritmo impiega $\Theta(n)$ sia per visitare l'intero albero ed impostare le altezze che per visitare l'intero albero per poi impostare i fattori di bilanciamento; quindi ha costo $\Theta(n)$.

Capitolo 2

Algoritmi su Grafi

2.1 Visitare i Grafi

Esercizio 2.1.1 #Albero #DFS

Dato un grafo $G = (V, E)$ non orientato, scrivere una procedura che verifichi se G è un albero. Dare la definizione di albero. Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Si definisce Albero un grafo composto da una sola componente connessa e senza cicli.

Elaborazione Soluzione: Possiamo vedere se il grafo G è un albero tramite l'algoritmo DFS verificando che consista di una sola componente connessa e che non abbia archi *backward*. Eseguiamo quindi una DFS modificata su G alla ricerca di archi all'indietro e contiamo le componenti connesse.

Si ricorda che all'interno di un grafo non orientato abbiamo 2 tipologie di archi:

1) **Dell'Albero**: Sono gli archi dell'albero di copertura generato dalla DFS. Si hanno quando il vertice raggiunto è bianco.

2) **All'Indietro**: Collegano un vertice ad un suo antenato, quindi si hanno quando il vertice raggiunto è grigio e non è il padre del nodo visitato. Infatti, in un grafo non orientato tutti gli archi sono visitati due volte, una volta per ciascuna direzione.

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Liste di Adiacenza.

```
ModDFS(Grafo G)
for ogni u ∈ G.V do
    u.colore = WHITE
    u.π = NULL
bool noCiclo = True
int card = 0
// Se card = |V| dopo una sola invocazione di ModDFSVisit, abbiamo una sola
componente connessa
noCiclo, card = ModDfsVisit(G, 1, True, 0)
// La Visita inizia dal vertice memorizzato in posizione 1
if noCiclo == True and card = |V| then
    return: True
else
    return: False
```

```
ModDfsVisit(Grafo G, int u, bool noCiclo, int card)
u.colore = GRAY
for ogni v ∈ Adj(u) do
    if v.colore == WHITE then
        card++
        v.π = u
        ModDfsVisit(G, v, noCiclo, card)
    else
        // Grafo non orientato, c'è un ciclo solo se il nodo raggiunto v è grigio ed è
         diverso dal padre di u
        if v.colore = GRAY And v ≠ u.π then
            return: (noCiclo = False, card)
u.colore = BLACK
return: (noCiclo, card)
```

Lancio dell'algoritmo: ModDfsVisit(G, u, $false$, 0)

Complessità: Il caso pessimo si verfica quando G è un albero, perchè dobbiamo visitarlo tutto prima di concludere. In tal caso, eseguiamo la DFS su ogni vertice, quindi il costo è quello di una DFS del grafo G: $O(V + E)$. Attenzione si possono anche toccare rei vertici neri, ma l'arco che li raggiunge è già stato battezzato in T o in B e quindi non ci interessano. **Perchè non ci sono archi forward e archi cross?**

Esercizio 2.1.2 #BFS

Dato un grafo $G = (V, E)$ orientato, non pesato, e due vertici $s, t \in V$, trovare la distanza da s a t in G.

- Dare la definizione di distanza
- Dare lo pseudo codice dell'algoritmo
- Discutere la complessità dell'algoritmo proposto.

Si definisce distanza da s a t in un grafo orientato e non pesato la lunghezza (numero di archi) del cammino più breve da s a t.

Elaborazione Soluzione: L'algoritmo è dunque una modifica della procedura BFS che termina quando il vertice t è inserito in coda.

```
Mod-BFS(Grafo G, Vertice s, Vertice t)
// Inizializzazione
Coda Q = ∅
for ogni v ∈ G.V do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
d(s) = NULL
ENQUEUE(Q, s)
trovato = FALSE
color(s) = GRAY
while Q ≠ NULL and trovato == FALSE do
    u = DEQUEUE(Q)
    for ogni v ∈ Adj(u) do
        if color(v) == WHITE then
            if v == t then
                trovato = TRUE, d(v) = d(u) + 1
            else
                π(v) = u, color(v) = GRAY, d(v) = d(u) + 1
                ENQUEUE(Q, v)
    color(u) = BLACK
return: d(t)
```

Complessità e Correttezza:
La complessità in tempo dell'algoritmo è:

- $O(V + E)$ se G è rappresentato da liste/vettore di adiacenza
- $O(V^2)$ se G è rappresentato da matrice di adiacenza

L'unica differenza è nell'implementazione del commando FOR OGNI $v \in Adj(u)$, che nel caso della matrice delle adiacenze diventa FOR $i = 1, \ldots, |V|$

Sappiamo che l'algoritmo è corretto per le proprietà della visita in ampiezza. Quando troviamo t per la prima volta lo abbiamo trovato con il cammino più breve che parte da s.

Perchè BFS trova la distanza minima?

Esercizio 2.1.3

#Sorgente #AlberoCopertura #EtichettamentoArchi #VettoreAdiacenze #DFS

Dato un grafo $G = (V, E)$ orientato, dare lo pseudo-codice della Depth First Search (DFS) che restituisce un albero di copertura di G radicato in s e restituisce anche il tipo di ciascun arco in E. Studiare la complessità in tempo dell'algoritmo proposto; si dica se il tipo degli archi dipende dalla sorgente s.

Elaborazione Soluzione: Scriviamo la DFS in modo che essa etichetti i tipi di archi presenti nel grafo $G = (V, E)$.

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Vettore di Adiacenza.

```
DFS(Grafo G)
for ogni u ∈ G.V do
    u.colore = WHITE
    u.π = NULL
time = 0
for ogni u ∈ G.V do
    if u.colore == WHITE then
        time = DfsVisit(G, u, time)

DfsVisit(Grafo G, Vertice u, int time)
time = time + 1
u.d = time
u.coloreGRAY
for [G.V[u]] ≤ i ≤ [G.V[u + 1] − 1] do
    v = G.E[i]
    if v.colore == WHITE then
        v.π = u
        G.E[i].type = "Albero" // Supponiamo in E ci sia il campo type per il tipo di
          arco
        time = DfsVisit(G, v, time)
    else
        if v.colore == GRAY then
            G.E[i].type = "Indietro"
        else
            if u.d < v.d then
                G.E[i].type = "Avanti"
            else
                G.E[i].type = "Trasversale"
u.colore = BLACK
time = time + 1
u.f = time
return: time
```

Complessità: Il costo è quello di una DFS: $O(V + E)$.

Per concludere, l'esercizio chiedeva di discutere se il tipo degli archi dipende dalla sorgente s. L'albero di copertura dipende dalla sorgente. Basta osservare che in funzione della sorgente s da cui lanciamo la DFS, i tempi di scoperta varieranno. Da questa osservazione, possiamo concludere che il tipo di arco dipende dal nodo sorgente s.

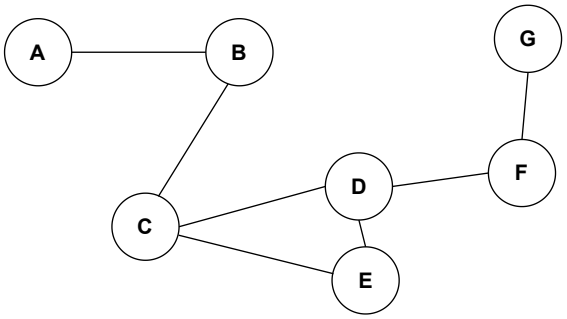
Esercizio 2.1.4 #BFS #DiametroGrafo

Dato un grafo connesso $G = (V, E)$ non orientato, memorizzato tramite liste di adiacenza, trovarne il diametro (la distanza massima fra tutte le coppie dei vertici in G).

- Dare lo pseudo codice dell'algoritmo.

- Discutere la complessità dell'algoritmo proposto.

Esempio:



Diametro: 5

$P_1 : A \to B \to C \to D \to F \to G$

Q: Perchè il diametro non ha valore 6?

$P_2 : A \to B \to C \to E \to D \to F \to G$

A: il cammino P_2 da A a G non è il cammino minimo.

Elaborazione Soluzione: Il diametro è la distanza massima fra le distanze delle coppie dei nodi in G. Formalmente:

$$diam(G) = max_{(u,v)\in V^2}\{dist(u,v)\}$$

Poiché si parla di distanza, sicuramente la visita in ampiezza BFS ci sarà utile. Tuttavia la BFS, fissata la sorgente $s \in V$, $BFS(G,s)$ calcola la distanza di tutti i vertici raggiungibili da s in G. Formalmente, BFS calcola

$$dist(s,v) \quad \forall\ v \in V$$

Pertanto, per calcolare il diametro di un grafo G possiamo sfruttare la visita in ampiezza per calcolare tutti cammini minimi **da ogni sorgente**, quindi per ogni nodo del grafo, a tutti i nodi raggiungibili da essa e memorizzare il massimo cammino computato:

$$\forall s \in V : compute\{dist(s,v)\ \forall\ v \in V\}$$

Realizziamo una versione della BFS che viene eseguita per ogni vertice del grafo. Restituiamo non solo il diametro (come richiesto dall'esercizio) ma anche una coppia la cui distanza è pari al diametro,

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Liste di Adiacenza.

```
get_diametro(Grafo G)
max = −∞, pair_1=pair_2=-1
for ogni v ∈ G.V do
    Coda Q = ∅
    // re-inizializziamo le distanze
    dist[1, ..., |V|] = −1
    dist[v] = 0; // Quella del primo vertice è 0
    color[1, ..., |V|] = WHITE // inizializziamo i colori
    Q.ENQUEUE(v)
    while Q ≠ ∅ do
        u = Q.DEQUEUE()
        for ∀k ∈ G.Adj[u] do
            if k.color == WHITE then
                k.color = GRAY
                dist[k] = dist[u] + 1
                if dist[k] > max then
                    max = dist[k], pair_1 = v, pair_2 = k
                Q.ENQUEUE(k)
        u.color = BLACK
return: max, pair_1, pair−2
```

Complessità: Dato che il primo **for** scorre tutte i nodi in V, abbiamo una BFS ripetuta per ogni nodo del grafo G. Segue che il costo dell'algoritmo è $O(V \cdot (V + E))$; ricordiamo che il grafo è memorizzato con liste di adiacenza.

Esercizio 2.1.5 #DFS #ComponenteConnessa

Dato un grafo $G = (V, E)$ non orientato, trovare la componente connessa di cardinalità massima.

- Dare la definizione di componente connessa;
- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Una componente connessa di un grafo è un insieme di vertici che hanno la stessa visibilità, ossia per ogni coppia di vertici della componente connessa esiste un cammino che li connette. Una componente connessa non è estensibile, ossia è un insieme massimale con la proprietà che i suoi vertici sono mutuamente raggiungibili.

Elaborazione Soluzione: L'algoritmo qui proposto è una versione modificata della DFS che tiene traccia della componente connessa trovata di dimensione maggiore.

Pseudocodice:

DFS(**Grafo** G)
for *ogni* $u \in G.V$ **do**
  $u.colore =$ WHITE
  $u.predecessore =$ NULL
$sizecc = 0$
$rootcc = 0$
Set $Sol = \emptyset$
for *ogni* $u \in G.V$ **do**
  if $u.colore ==$ WHITE **then**
    Set $tmpSol = \{u\}$ // Per ora c'è solo u
    int $tmpSize = 1$ // La sua dimensione della componente connessa
    DfsVisit($G, u, *tmpSol, *tmpSize$) // * è passaggio per riferimento
    if $tmpSize > sizecc$ **then**
      $rootcc = u$
      $sizecc = tmpSize$
      $Sol = tmpSol$
  $color(u) =$ BLACK
return: ($rootcc$, $sizecc$, Sol)

DfsVisit(**Grafo** G, **Nodo** u, **Set Pointer *** $tempSol$, **int Pointer *** $tmpSize$)
$u.colore =$ GRAY
for *ogni* $v \in Adj[u]$ **do**
  if $v.colore ==$ WHITE **then**
    $color(v) =$ GRAY
    $tmpSize++$
    $tmpSol = tmpSol \cup v$
    DfsVisit(G, v, $*tmpSize$, $*tmpSol$)
$color(u) =$ BLACK
Complessità:

- $O(V + E)$ se si utilizza una lista/vettore delle adiacenze
- $O(V^2)$ se si utilizza una matrice delle adiacenze

Esercizio 2.1.6 #BFS #Eccentricità

Dato un grafo $G = (V, E)$ non orientato, non pesato (o con pesi costanti) e dato un nodo v, trovare la sua eccentricità in G (i.e., $e(v) = \max\{d(v, u)|u \in V\}$).

- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

L'eccentricità di un nodo in un grafo è una misura della massima distanza tra quel nodo e qualsiasi altro nodo nel grafo.

Elaborazione Soluzione: Possiamo risolvere questo problema utilizzando l'algoritmo Breadth-First Search (BFS). La BFS ci permette infatti di trovare i percorsi più brevi tra un nodo di partenza e tutti gli altri nodi nel grafo in termini di archi.
Eseguendo BFS a partire da un certo nodo $v \in V$ dato in input e trovando la distanza massima da qualsiasi altro nodo, possiamo determinare l'eccentricità del nodo v ritornando il nodo u

con la distanza massima da v, i.e., $max\{d(v,u)|u \in V\}$.

Pseudocodice:

```
find_eccentricity(Grafo G, vertex s)
for ogni v ∈ G.V − s do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
max_d = 0
d(s) = NULL
color(s) = GRAY
π(s) = NULL
Coda Q = ∅
ENQUEUE(Q, s)
while Q ≠ NULL do
    u = DEQUEUE(Q)
    for ogni v ∈ G.Adj(u) do
        if color(v) == WHITE then
            d(v) = d(u) + 1
            if max_d < d(v) then
                max_d = d(v)
            color(v) = GRAY
            π(v) = u
            ENQUEUE(Q, v)
    color(u) = BLACK
return: max_d
```

Complessità:

- $O(V + E)$ se G è rappresentato da liste/vettore di adiacenza
- $O(V^2)$ se G è rappresentato da matrice di adiacenza

Esercizio 2.1.7 #DFS

Sia $G = (V, E)$ un grafo non diretto e non connesso: trovare quanti archi si debbono aggiungere per rendere connesso G.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Sappiamo che gli algoritmi BFS e DFS possono essere utilizzati per la ricerca di componenti connesse. Per connettere un grafo sconnesso servono tanti archi quanti il numero di componenti connesse meno uno.

Pseudocodice:

```
DFS-Comp-Conn(Grafo G)
for ogni u ∈ G.V do
    color(u) = WHITE
int i = 0 // numero di componenti
for ogni u ∈ G.V do
    i++
    if color(u) = WHITE then
        DfsVisit(G, u)
return: i − 1
DfsVisit(Grafo G, Vertice u)
color(u) = GRAY
for ogni v ∈ Adj[u]} do
    if color(v) = WHITE then
        v.predecessore = u
        DfsVisit(G, v)
color(v) = BLACK
```

Complessità: Stiamo qui eseguendo una normale DFS, quindi il costo è quello, $\Theta(V+E)$.

2.2 Operazioni sui Grafi

Esercizio 2.2.1 #DFS #OrdinamentoTopologico #GrafoDirettoAciclico

Dato un grafo $G = (V, E)$ orientato ed una sorgente r dalla quale si raggiungono tutti i vertici in V, restituire, se esiste, l'ordine dei vertici che soddisfa l'ordinamento topologico.

- Dare la definizione di ordinamento topologico;
- Dire sotto quali condizioni esiste l'ordinamento topologico;
- Dare lo pseudo codice dell'algoritmo ('algoritmo deve restituire False se l'ordinamento non è possibile);
- Discutere la complessità dell'algoritmo proposto.

Elaborazione Soluzione: E' possibile trovare l'ordinamento topologico solo se il grafo è un Directed Acyclic Graph (DAG). L'ordinamento topologico garantisce che, esiste una linearizzazione dei vertici tale che posti i vertici sulla linea per ciascun arco il vertice da cui esso parte precede nell'ordinamento il vertice destinazione dell'arco stesso.

L'algoritmo che risolve il nostro esercizio, SORT-TOP, ritorna la lista L dei vertici in ordine decrescente di fine visita (il pozzo del nostro grafo ha il minimo tempo di visita). L'algoritmo ritorna FALSE se non esiste l'ordinamento topologico, ossia se trova un arco di tipo backward. In tal caso, esiste un ciclo e pertanto il grafo non è un DAG.

Pseudocodice:

```
DFS(Grafo G)
for ogni u ∈ G.V do
    color(u) = WHITE
    u.predecessore = NULL
List L = ∅ // Lista ordinata dei vertici visitati
DFSVisit(G, u, 0, L)

DFSVisit(Grafo G, Vertice u, int time, List L)
time = time + 1
u.d = time
color(u)GRAY
for ogni v ∈ Adj[u]} do
    if color(v) = WHITE then
        v.predecessore = u
        DFSVisit(G, v, time, L)
    else
        if color(v) = GRAY then
            // Abbiamo un arco all'indietro, quindi non è un DAG
            return: False
color(v) = BLACK
time = time + 1
u.f = time
INSERT(u, L) // Inserimento in testa
return: L
```

sort-top(**Grafo** G)
result = DFS(G)
if *result* $\neq$ FALSE **then**
  return: *result*
  // Si ritorna la lista dei vertici ordinati topologicamente
else
  return: "Vi è un Ciclo"

Complessità:

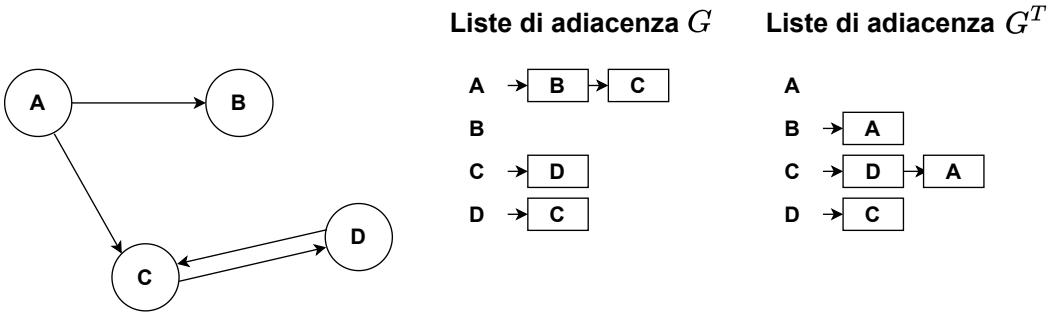
- Abbiamo la complessità della DFS su lista di adiacenza: $O(V + E)$
- Poichè dal vertice r si raggiungono tutti gli altri vertici è di fatto sufficiente invocare la DFSVISIT a partire da r.

Esercizio 2.2.2 #ListeAdiacenza #GrafoTrasposto #GrafoDiretto

Dato un grafo $G = (V, E)$ orientato, memorizzato tramite liste di adiacenza, trovarne il trasposto G^T.

- Dare lo pseudo codice dell'algoritmo.
- Discutere la complessità dell'algoritmo proposto.

Esempio:



Elaborazione Soluzione: Per ogni coppia di nodi $v, u \in V$ viene invertita la relazione che li lega.

Pseudocodice
Reverse_List_Adj(**Grafo** G)
inizializza G^T
for $\forall v \in G.V$ **do**
  for $\forall u \in G.adj[v]$ **do**
    $G^T.adj[u]$.APPEND(v)
return: G^T

Funzionamento:

- per **inizializza** G^T si intende la creazione di un dizionario le cui chiavi sono i vertici della grafo G, e i cui campi sono vuoti; ovvero ogni lista delle adiacenze ha la sola testa inizializzata.
- La funzione APPEND() viene utilizzata per inserire in testa l'ultimo elemento che si è

letto nella rispettiva lista.

Complessità: Il primo ciclo **for** itera sopra ogni vertice di $G.V$, segue che il suo costo è $O(V)$. Il secondo ciclo **for** visita per ogni nodo tutti i suoi archi uscenti esattamente una volta; pertanto ci costa $O(E)$.
Segue che la complessità finale dell'algoritmo proposto è $O(V + E)$.

Esercizio 2.2.3 #GrafoPotenza #DFS

Dato un grafo $G = (V, E)$ diretto, non pesato e memorizzato con matrice di adiacenza, e dato un intero k, si vuole costruire il grafo potenza G^K; quindi un grafo che ha gli stessi vertici di G e un arco per ogni coppia di nodi a distanza **al più** k in G.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione:
Definiamo:

$$L^k_{(i,j)} = \bigvee_{h=1}^{n} \left(L^{k-1}_{(i,h)} \wedge L^1_{(h,j)} \right)$$

Tale formula ci permette di determinare se tra due nodi i e j nel grafo L vi è un cammino di lunghezza k, nella posizione $L^k_{(i,j)}$ della matrice avremo TRUE se esiste l'arco nel grafo potenza e FALSE diversamente.

Supponiamo che la matrice di adiacenza iniziale data abbia anche essa TRUE dove vi è un arco tra due nodi e FALSE dove non vi è.

Per risolvere il nostro problema dobbiamo quindi calcolare tutti i grafi potenza fino a k e sovrapporli per trovare i cammini di lunghezza al più k; quindi fare l'unione di tutti i grafi G^1, G^2, $\cdots$, G^k a formare G^K.

Pseudocodice:

```
GrafoPotenza(Matrix G[n × n], int k)
// G è matrice di adiacenza con TRUE e FALSE
G^1[n × n] = G
for m = 2 to k do
    G^m = OPERATION(G^(m−1), G^1)// le matrici sono passate per riferimento
return: G^k

Operation(Matrix G_{m−1}[n × n], Matrix G_1[n × n])
Matrix G_m[n × n] // Creo una nuova matrice di dimensione n × n
for i = 1 to n do
    for j = 1 to n do
        temp = FALSE
        h = 1
        while h ≤ n and temp == FALSE do
            temp = temp or (G_{m−1}[i, h] and G_1[h, j])
            h++
        G_m[i, j] = temp
return: G_m
```

Complessità: L'algoritmo equivale a fare k volte una moltiplicazione di matrici $n \times n$. Quindi ha costo $O(k \cdot n^3)$

2.3 Cammini Minimi da Sorgente Singola

Esercizio 2.3.1 #DagShortestPaths #Ottimalità

Dato un grafo $G = (V, E)$ diretto aciclico e pesato $w : E \to \mathbb{N}$, dato un intero $k \in \mathbb{N}$, e data una sorgente $s \in V$, trovare i vertici in V che distano al più k da s.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.
Discutere l'ottimalità della soluzione proposta.

Elaborazione Soluzione: Il grafo è pesato. I pesi non hanno limitazioni. Per calcolare gli shortest paths, possiamo applicare l'algoritmo di Bellman Ford o quello di Floyd-Warshall. Poiché sappiamo che il grafo è diretto ed aciclico, possiamo eseguire la varante di Bellman Ford che considera gli archi una sola volta (e non $|V|$). Chiamiamo tale variante Dag-Shortest-Paths. L'algoritmo assume di rilassare i vertici nell'ordine dato dal sort-topologico del grafo. Dato che a noi interessano i vertici a distanza al più k, una volta che l'algoritmo ha finito di trovare le distanze minime dalla sorgente scorriamo tutti i nodi e selezioniamo quelli a distanza minore od uguale a k.

Pseudocodice:

```
Dag-Shortest-Paths-Dist-k(Grafo G, Funzione Peso w, Sorgente s, int k)
Lista sol = ∅ // Qui va la soluzione
Lista L = TOPOLOGICAL-SORT(G) // DFS dove ordinano nodi per tempo fine visita
// L'inizializzazione:
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
s.d = 0
for ogni vertice u ∈ G.V presi secondo l'ordine topologico do
    for ogni vertice v ∈ G.Adj[u] do
        // Il rilassamento:
        if v.d > u.d + w(u, v) then
            v.d = u.d + w(u, v)
            v.predecessore = u
// cerchiamo quelli di nostro interesse
for ogni vertice u ∈ G.V do
    if v.d ≤ k then
        INSERT(v, sol)
return: sol
```

Complessità: L'algoritmo Dag-Shortest-Paths-Dist-k sfrutta il fatto che il grafo sia un DAG, infatti il sort topologico permette di fare la relax una sola volta per ogni arco; questo perché l'ordinamento topologico ci fornisce un cammino semplice che va da un nodo a tutti i suoi successivi.

Il costo di Dag-Shortest-Paths-Dist-k è $O(V + E)$, che è ottimo; infatti il grafo rappresentato con liste di adiacenza ha dimensione $(V + E)$; lo scorrere tutti i vertici per trovare quelli a distanza al più k ci costa $O(V)$, quindi il costo resta asintoticamente $O(V + E)$.

Esercizio 2.3.2 #DagShortestPaths #Ottimalità

Dato un grafo orientato e aciclico $G = (V, E)$ pesato $w : E \to \mathbb{R}$ e data una sorgente $s \in V$, progettare un algoritmo per trovare i cammini di costo minimo da s verso tutti gli altri vertici in V.

Discutere la soluzione proposta e discutere la complessità in tempo della soluzione proposta. La soluzione proposta è ottima? Motivare la risposta.

Elaborazione Soluzione: Eseguiamo l'algoritmo DAG-SHORTEST-PATHS che effettua un sort-topologico per poter fare la relax una sola volta di tutti gli archi seguendo l'ordinamento topologico.

Pseudocodice:

```
Dag-Shortest-Paths(Grafo G, Funzione Peso ω, Sorgente s)
List L = Topological-Sort(G) // DFS collezionando nodi per tempo fine visita
// L'inizializzazione:
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
s.d = 0
for ogni vertice u ∈ G.V presi secondo l'ordine topologico in L do
    for ogni vertice v ∈ G.Adj[u] do
        // Il rilassamento:
        if v.d > u.d + ω(u, v) then
            v.d = u.d + ω(u, v)
            v.predecessore = u
```

Complessità: Il costo di DAG-SHORTEST-PATHS è $O(V + E)$ che è ottimo; infatti il grafo rappresentato con liste di adiacenza ha dimensione $(V + E)$.

Esercizio 2.3.3 #Bellman-Ford

Dato un grafo $G = (V, E)$, orientato e pesato con pesi arbitrari, trovare se esiste un ciclo di costo negativo raggiungibile da una sorgente s.

- Descrivere l'algoritmo che si propone
- Darne la descrizione in pseudocodice
- Discuterne la complessità in tempo dell'algoritmo.

Elaborazione Soluzione: Possiamo utilizzare l'algoritmo di Bellman-Ford. Esso calcola i cammini minimi da sorgente singola per tutti i nodi raggiungibili.

Pseudocodice:

Bellman-Ford(**Grafo** G, **Funzione Peso** ω, **Sorgente** s)
// L'inizializzazione:
for *ogni vertice* $v \in G.V$ **do**
  $v.d = +\infty$
  $v.predecessore =$NULL
$s.d = 0$
for $i = 1$ **to** $|G.V| - 1$ **do**
  for *ogni arco* $(u, v) \in G.E$ **do**
    // Il rilassamento:
    if $v.d > u.d + \omega(u, v)$ **then**
      $v.d = u.d + \omega(u, v)$
      $v.predecessore = u$
for *ogni arco* $(u, v) \in G.E$ **do**
  if $v.d > u.d + \omega(u, v)$ **then**
    return: FALSE
return: TRUE

Complessità: Bellman-Ford implementato con liste di adiacenza ha costo $O(V \cdot E)$ dato dal rilassare tutti gli archi del grafo, $|E|$, per $|V|$ volte.

Esercizio 2.3.4 #Dijkstra

Dato un grafo $G = (V, E)$ diretto e pesato $\omega : E \to N^+$, data una sorgente $s \in V$, trovare il vertice u' tale che il cammino da s a u' sia quello di peso $d(s, u')$ minimo in G. Ossia $d(s, u') = \min_{u \in V} d(s, u)$.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice. Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Abbiamo un grafo pesato con pesi non negativi dove vogliamo trovare il vertice più vicino alla sorgente. Possiamo eseguire l'algoritmo di Dijkstra per trovare le distanze di ogni vertice dalla sorgente e poi scorrerli tutti per trovare quello di nostro interesse.

Pseudocodice:

```
Dijkstra-Most-Close(Grafo G, Funzione Peso ω, Nodo Sorgente S)
// Inizializzazione
for ogni vertice v ∈ G.V do
    v.d = +∞
    v.predecessore = NULL
S.d = 0
Lista L = ∅
Min-Heap Q = ∅
for ogni vertice v ∈ G.V do
    INSERT(Q, v)
trovato = FALSE
while IS-EMPTY(Q) == FALSE and trovato == FALSE do
    u = EXTRACTMIN(Q)
    for ogni vertice v ∈ G.adj[u] do
        if v.d > u.d + ω(u, v) then
            v.d = u.d + ω(u, v)
            v.predecessore = u
    ENQUEUE(L, u)
Nodo n // Qui mettiamo la soluzione
int min = +∞ // Distanza minima
for ogni vertice v ∈ G.V do
    if v.d < min then
        min = v.d
        n = v
return: n
```

Complessità: L'algoritmo di Dijkstra implementato con un min-heap ha costo $O((E + V) \cdot \log V)$ che è maggiore del costo $O(V)$ usato per la ricerca del nostro nodo d'interesse. Poi si cerca u' cercando il vertice con distanza minima.

Attenzione: Nonostante l'algoritmo sia corretto, non sfrutta il fatto che i nodi ritornati da Dijkstra sono in ordine crescente di distanza dalla sorgente. Pertanto u' è il nodo in G con peso minimo uscente dalla sorgente! Quindi la complessità è $O(|Adj(s)|)$ ossia il grado del nodo sorgente.

2.4 Alberi di Copertura

Esercizio 2.4.1 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo connesso $G = (V, E)$ non orientato, i cui archi hanno pesi interi minori od uguali a 3, ossia $w : E \to \{1, 2, 3\}$, trovare l'albero di copertura T_G di costo minimo. Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Il grafo $G = (V, E)$ è caratterizzato dal fatto che la funzione peso è definita come segue: $w : E \to \{1, 2, 3\}$.

Sappiamo che l'algoritmo di Kruskal per il calcolo dell'albero di copertura minimo seleziona ad ogni passo gli archi più leggeri, assicurandosi di non formare cicli. Inizia con un insieme vuoto di archi e poi ordina gli archi in base ai loro pesi. Quindi, itera attraverso gli archi, aggiungendo all'insieme quelli che non creano cicli. L'algoritmo continua finché l'albero di copertura minimo contiene $|V| - 1$ archi.

Dato che l'insieme dei pesi è finito e caratterizzato da soli valori interi, possiamo ottenere un miglioramento delle prestazioni di Kruskal utilizzando l'algoritmo di ordinamento COUNTING SORT, con il quale si ordinano gli archi in tempo lineare.

Pseudocodice:

```
KruskalCountingSort(Graph G)
List E = Archi di G.E Ordinati in Ordine Decrescente Usando COUNTING-SORT
List T = ∅ // Lista per gli archi dell'albero di copertura minimo
for ogni v in G.V do
    MAKE-SET(v)
for ogni arco (u, v) in E preso nell'ordine dalla lista E do
    if FIND(u) ≠ FIND(v) then
        T = T ∪ (u, v)
        UNION(u, v)
return: T
```

Complessità: Il tempo di esecuzione di Kruskal è dato dall'uso di MERGE-SORT per ordinare $G.E$ e dall'utilizzo della struttura dati ad Insiemi Disgiunti. Per le strutture dati, noi esprimiamo la complessità in tempo come numero di operazione Find e numero di operazioni Union. L'uso del COUNTING-SORT migliora il tempo dell'ordinamento, perché porta il costo dell'ordinamento a $O(E)$.

Il tempo di esecuzione asintotico pertanto è: $O(E + 2|E|T_{\text{FIND}} + (|V| - 1)T_{\text{UNION}})$.

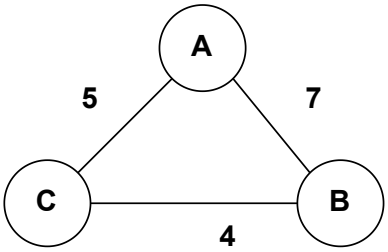
Esercizio 2.4.2 #AlberoCoperturaCostoMinimo #AlberoCoperturaCamminimiCostoMinimo

Si consideri il grafo non orientato $G = (V, E)$ e pesato $w : E \to N$ dove $V = \{A, B, C\}$ e $E = (A, B), (A, C), (B, C)$ (ossia un ciclo con 3 vertici).

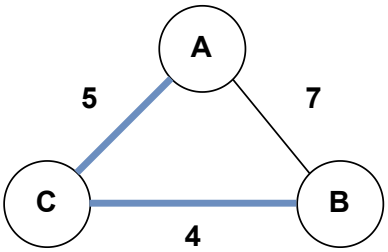
- Determinare i pesi sugli archi in modo che l'albero dei cammini di costo minimo (Albero dei Cammini Minimi) radicato nel vertice A sia diverso dall'albero di copertura di costo minimo (Albero di Copertura Minimo) del grafo G;
- Illustrare i due alberi trovati

Proponiamo qui una possibile soluzione dell'esercizio:

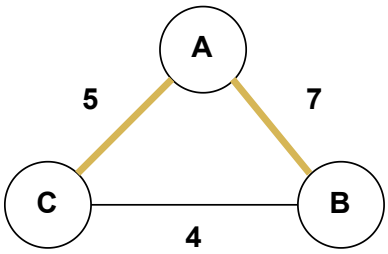
Soluzione:



MST



Cammini Minimi



Esercizio 2.4.3 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo non diretto $G = (V, E)$ i cui archi hanno peso 1 o 10, ossia $w : E \to \{1, 10\}$, la Prof. Fati-Na, ha proposto il seguente algoritmo per calcolare l'albero di copertura di costo minimo:

MST-UltraShort(**Grafo** G, **Funzione Peso** w)
Let $E' \subseteq E$ is the set of edges with $w(e) = 1$
$T = DFS(G' = (V, E'))$
return: T

L'algoritmo proposto è corretto?
Motivare la risposta: se non è corretto dare un controesempio. Eventualmente correggere l'algoritmo. Discutere la complessità dell'algoritmo, eventualmente corretto.

L'algoritmo non è corretto. L'eliminazione di archi di costo 10 potrebbe disconnettere dei vertici dal grafo; ad esempio avendo un vertice i cui archi incidenti hanno solo costo 10. Inoltre, archi di costo 1 potrebbero formare cicli.

Algoritmo Corretto:

KruskalCountingSort(**Graph** G)
List E = Archi di $G.E$ Ordinati in Ordine Decrescente Usando **Counting-Sort**
List $T = \emptyset$ // Lista con gli Archi dell'albero di copertura minimo
for *ogni* v *in* $G.V$ **do**
  MAKE-SET(v)
for *ogni arco* (u, v) *in* E *preso nell'ordine dalla lista* E **do**
  if FIND(u) $\neq$ FIND(v) **then**
    $T = T \cup (u, v)$
    UNION(u, v)
return: T

Complessità: Sapendo che i pesi degli archi sono nell'insieme $\{1, 10\}$ possiamo applicare l'algoritmo di Kruskal con counting-sort per migliorarne il costo; infatti la complessità temporale dell'algoritmo di Kruskal è data principalmente dall'ordinamento dei vertici, che qui è $O(E)$. Il tempo di esecuzione asintotico pertanto è $O(E + 2|E|T_{\text{FIND}} + (|V| - 1)T_{\text{UNION}})$.

Esercizio 2.4.4 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo non orientato e pesato $G = (V, E)$ e il suo albero di copertura di costo minimo T, si consideri il grafo ottenuto aggiungendo l'arco pesato $e = (u, v)$, ossia $G' = (V, E \cup (u, v))$ (i vertici sono invariati). Sia T' l'albero di copertura di costo minimo per G'. Dire per ogni affermazione se è vera o falsa (giustificando la risposta):

- $e \in T'$ se il peso $w(e)$ è il peso minimo degli archi in G
- $e \in T'$ se il peso $w(e)$ è il peso massimo degli archi in G'
- T' e T differiscono al più per 1 arco

Dare lo pseudocodice di un algoritmo che calcola T' conoscendo G' e T.
Discutere la complessità in tempo dell'algoritmo proposto quando i pesi degli archi sono interi compresi nell'intervallo $[0, 10]$.

Risposte alle Domande. Si fa riferimento solo alla costruzione di T' a partire da T. Si noti che il nuovo arco forma un ciclo C.

- Vero perché sarà il primo arco colorato di blue dall'algoritmo. Solo se C contiene tutti archi del peso uguale a $w(e)$ allora e non è inserito in T'.
- Falso. L'arco di costo massimo $w(e)$ è escluso. Attenzione, si noti che i vertici non sono stati modificati.
- Falso. Ecco un caso in cui T e T' differiscono per due archi. $w(e)$ è l'arco di costo minimo, e entra in T', ma un altro arco esce perchè si forma un ciclo.

Elaborazione Soluzione: Sappiamo che T è un albero di copertura minimo per G, poiché G' differisce da G solo per la aggiunta dell'arco e, possiamo semplicemente aggiungere e a T, creando T_e, per poi fare in modo che T_e resti un albero di copertura minimo, così da avere T'.

Dato l'arco $e = (u, v)$, seguendo il campo padre in T troviamo gli archi che formeranno il ciclo con e. Escludiamo l'arco di peso massimo, aggiungiamo e e dovremmo correggere i padri.

2.5 Cammini Minimi tra Tutte le Coppie

Esercizio 2.5.1 #MetodoTropicale

Dato un grafo $G = (V, E)$ connesso, pesato $w : E \to R$, ed orientato, data una sorgente $s \in V$, trovare il costo dei cammini minimi da s a tutti gli altri vertici in V di lunghezza al più 3. Dare lo pseudo-codice dell'algoritmo, calcolare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Abbiamo un grafo orientato e pesato con pesi qualsiasi (quindi potenzialmente anche negativi) avente una sorgente s dalla quale vogliamo trovare tutti i cammini minimi di lunghezza ≤ 3 verso ogni altro nodo del grafo; inoltre il grafo potrebbe avere dei cicli di costo negativo, visto che non è specificato il contrario; eventualmente individuabili con un esecuzione di Bellman-Ford di costo $O(V \cdot E)$.

Infatti per eseguire l'algoritmo sotto descritto si deve supporre che non vi siano cicli di costo negativo.

L'unico algoritmo che conosciamo per la ricerca di cammini minimi da sorgente singola nel contesto sopra descritto è Bellman-Ford. Ma Bellman-Ford non risolve il nostro problema, infatti i cammini dalla sorgente agli altri vertici che trova sono di lunghezza non specificata, mentre a noi servono quelli di lunghezza massima 3. Quindi ad esempio se tra la sorgente s ed il vertice v abbiamo un cammino di lunghezza 5, di costo 4, ed un cammino di lunghezza 3 con costo 7 troverà il primo, mentre a noi serve il secondo.

Dobbiamo sfruttare il Metodo Tropicale per la ricerca di cammini tra tutte le coppie di un grafo. Infatti questo algoritmo permette di trovare i cammini minimi tra tutte le coppie, di una certa lunghezza al più k, nella k-esima matrice da esso generata.

La soluzione al nostro problema sarà quindi nella terza matrice generata dall'algoritmo alla riga corrispondente al vertice s cioè la nostra sorgente.

Il Metodo Tropicale ha costo $O(n^4)$ ma poiché noi lo eseguiamo solo fino alla terza matrice e non fino alla n-esima, cioè quella matrice avente i cammini tra tutte le coppie di lunghezza al più n, il costo si riduce a $O(n^3)$.
Per ridurre ulteriormente il costo della nostra soluzione possiamo eseguire i calcoli non per l'intera matrice, ma solo per la riga corrispondente al vertice s; cioè l'unica di nostro interesse.

Pseudocodice:

Tropical-Single-Source(**Matrix** $W[n \text{ x } n]$) // W è la matrice di adiacenza con pesi
Array $R^1[n] = W[s]$ // Selezioniamo la riga corrispondente ad s
for $m = 2$ **to** 3 **do**
  // Ci serve il terzo vettore R^3
  $R^m = \text{EXTENDED-SHORTEST-PATHS}(R^{(m-1)}, W)$
return: R^3

Extended-Shortest-Paths(**Array** $R[n]$, **Matrix** $W[n \text{ x } n]$)
Matrix $H^1[n] = \{\infty, \cdots, \infty\}$ // Creo il nuovo vettore H di dimensione n pieno di ∞
for $i = 1$ **to** n **do**
  for $k = 1$ **to** n **do**
    $H[i] = \text{MIN}(H[i], R[k] + W[k, i])$
return: H

Complessità: Lavorando qui solo con il vettore corrispondente alla nostra sorgente il costo scende ad $O(n^2)$. Attenzione se esiste un ciclo di costo negativo di lunghezza al più 3 che include il vertice s, avremo un valore negativo sulla posizione $H(s, s)$.

Esercizio 2.5.2 #Floyd-Warshall

Sia $G = (V, E)$ un grafo i cui vertici sono identificati da 1 a $|V|$ e i cui archi sono pesati $w : E \to \mathbb{R}^+$. Trovare per ogni coppia di vertici in $V \times V$ il cammino di costo minimo che passa per

i soli vertici identificati da un numero pari.

Esempio: è ammissibile il cammino da 1 a 7 che passa per i vertici 2 e 6, ossia 1-2-6-7, mentre non è ammissibile il cammino da 1 a 7 che passa per i vertici 2 e 3, ossia 1-2-3-7, perché 3 è un vertice identificato da un numero dispari.

- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Poiché cerchiamo i cammini di costo minimo tra tutti i vertici di un grafo avente pesi positivi possiamo utilizzare l'algoritmo di Floyd-Warshall, che infatti lavora sui vertici. Si ricorda che per usare l'algoritmo di Floyd-Warshall si deve supporre che non vi siano cicli di costo negativo (Eventualmente trovabili con l'ausilio dell'algoritmo di Bellman-Ford). Essendo i pesi non negativi, non possono esistere cicli di costo negativo.

Qui la soluzione più immediata è una modifica dell'algoritmo di Floyd-Warshall, a fare in modo che esso consideri solo i vertici pari come vertici intermedi (dove $V = \{1, 2, \ldots, |V|\}$:

$$d_{u,v}^{(2k)} = \min\{d_{u,v}^{2(k-1)};\ d_{u,2k}^{2(k-1)} + d_{2k,v}^{2(k-1)}\}$$

L'equazione sopra riportata rappresenta l'equazione di ricorrenza per calcolare i cammini di costo minimo nel nostro problema (è una versione modificata di quella di Floyd-Warshall che tiene conto solo dei vertici pari '2k').

Inoltre definiamo:

- $D^{(0)}$: matrice delle adiacenze che rappresenta G modificato con ∞ se la coppia (i, j) non appartiene ad E.

$$D_{i,j}^{(0)} = \begin{cases} w(i,j) & \text{if } (i,j) \in E \text{ con } i \neq j \\ 0 & \text{if } i = j \\ +\infty & \text{if } (i,j) \notin E \end{cases}$$

- $\pi^{(0)}$: matrice delle adiacenze che rappresenta Π (cioè la matrice dei predecessori).

$$\pi_{i,j}^{(0)} = \begin{cases} \text{NULL} & \text{if } i = j \text{ or } (i,j) \notin E \\ i & \text{if } i \neq j \text{ and } (i,j) \in E \end{cases}$$

Pseudocodice:

Even-Floyd-Warshall(**Grafo** G)
// Abbiamo supposto di avere già definito $D^{(0)}_{i,j}$
for $k = 2$ **to** $\lfloor \frac{|V|}{2} \rfloor$ **step** 2 **do**
  for $i = 1$ **to** $|V|$ **do**
    for $j = 1$ **to** $|V|$ **do**
      $temp = D^{(k-2)}(i,k) + D^{(k-2)}(k,j)$
      if $D^{(k-2)}(i,j) > temp$ **then**
        $D^{(k)}(i,j) = temp$
        $\pi^{(k)}(i,j) = \pi^{(k-2)}(k,j)$ // Calcolo del padre
      else
        $D^{(k)}(i,j) = D^{(k-2)}(i,j)$
        $\pi^{(k)}(i,j) = \pi^{(k-2)}(i,j)$ // Calcolo padre per ricostruzione soluzione
return: $D^{\lfloor \frac{|V|}{2} \rfloor}$
Complessità:

- La complessità in tempo è $\frac{|V|}{2}|V|^2 = O(V^3)$
- La complessità in spazio è data dal fatto che si calcolano $\frac{|V|}{2}$ matrici per i costi dei cammini e $\frac{|V|}{2}$ matrici per tenere traccia dei cammini, tutte di dimensione $|V| \cdot |V|$; quindi abbiamo un costo totale di $O(V^3)$.

La ricostruzione dei cammini viene effettuata tramite l'algoritmo seguente, che viene evocato per ogni coppia di vertici di cui si vuole avere il cammino minimo che li connette:

ShortestPath(**Vertice** u, **Vertice** v, **Matrice Predecessori** π)
if $\pi[u,v] =$ NULL **then**
  return: $NoPaths$
List $path = \emptyset$
Node $x = v$ // Variabile locale per memorizzare i nodi
ENQUEUE($path, x$)
while $u \neq x$ **do**
  $x = \pi[u,x]$ // Percorriamo a Ritroso il Cammino Minimo tra u e v
  ENQUEUE($path, x$)
return: $path$

Indice delle Etichette

Albero, 5
AlberoBinarioRicerca, 1, 2
AlberoCopertura, 7
AlberoCoperturaCamminimiCostoMinimo, 21
AlberoCoperturaCostoMinimo, 21–23

Bellman-Ford, 18
BFS, 6, 8, 11

ComponenteConnessa, 10
Correttezza, 1

DagShortestPaths, 17
DFS, 5, 7, 10, 12, 14, 16
DiametroGrafo, 8
Dijkstra, 19

Ecccentricità, 11
EtichettamentoArchi, 7

Floyd-Warshall, 24

GrafoDiretto, 15
GrafoDirettoAciclico, 14
GrafoPotenza, 16
GrafoTrasposto, 15

Kruskal, 21–23

ListeAdiacenza, 15

MetodoTropicale, 24

OrdinamentoTopologico, 14
Ottimalità, 17

Sorgente, 7

VettoreAdiacenze, 7

Quest'opera è distribuita con licenza Creative Commons "Attribuzione – Non commerciale 4.0 Internazionale".

